

115 學年度資訊學科能力競賽臺南一中校內複選 試題本

競賽規則

1. 競賽時間：2026/09/24 13:00 ~ 17:00，共 4 小時。
2. 本次競賽試題共 7 題，每題皆有子任務。
3. 為了愛護地球，本次競賽題本僅提供電子檔，不提供紙本。
4. 每題的分數為該題所有子任務得分數加總；單筆子任務得分數為各筆繳交在該筆得到的最大分數。
5. 本次複選不提供記分板。
6. 除通訊題目以外其他題目的輸入皆為標準輸入。
7. 除通訊題目以外其他題目的輸出皆為標準輸出。
8. 所有輸入輸出請嚴格遵守題目要求，多或少的換行及空格皆有可能造成裁判系統判斷為答案錯誤。
9. 每題每次上傳間隔為 120 秒，裁判得視情況調整。
10. 所有試題相關問題請於競賽系統中提問，題目相關公告也會公告於競賽系統，請密切注意。
11. 如有電腦問題，請舉手向監考人員反映。
12. 如有如廁需求，須經過監考人員同意方可離場。
13. 不得攜帶任何參考資料，但競賽系統上的參考資料可自行閱讀。
14. 不得自行攜帶隨身碟，如需備份資料，請將資料儲存於電腦 D 槽。
15. 競賽中請勿交談。請勿做出任何會干擾競賽的行為。
16. 如需使用 C++ 的 `std::cin` 或 `std::cout` 可將以下程式碼插入 `main function` 以及將 `endl` 取代為 `'\n'` 來最佳化輸入輸出速度。唯須注意不可與 `cstdio` 混用。

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

A. 彩虹遊行

Problem ID: Rainbow_Parade

Time Limit: 2s

Memory Limit: 512MiB

家豪（Charhao）身為台南一中的三好學生，平時最喜歡翹課出來參加彩虹遊行。顧名思義，彩虹遊行指的是一大群彩虹小馬愛好者聚在一起的盛大活動。

今天的彩虹遊行在一座神奇的島嶼上舉辦，島上有 n 個彩虹小馬景點，編號為 $1 \sim n$ 。島嶼的入口位於 1 號景點，而開幕式場地則在 n 號景點。

家豪對這個島嶼一無所知，一個人走一定會迷路，所以他需要熱心助人的彩虹小馬學長 tw87 帶路。島上共有 m 條單向道路，第 i 條道路從景點 a_i 連向景點 b_i 。

由於 tw87 趕著去參加「彩虹小馬聖經研討會」，無法親自帶路，於是他決定畫一張地圖給家豪。地圖的繪製規則如下：

- 除了開幕式景點 n 之外，每個景點都必須**恰好選擇**一條從該點出發的單向道路，寫下下一個該去的景點。
- 為了避免地圖標示混亂，除了入口 1 號景點外，其餘每個景點在地圖上**恰好被一個景點指引**。
- 家豪必須能順利從入口 1 號景點沿著指引走到開幕式 n 號景點。但為了滿足 tw87 的惡趣味，對於**不在這條主線路徑上**的其他景點，tw87 刻意將它們設計成**若家豪不小心闖入，將會陷入無限繞圈圈的死循環中**。

每條道路 i 都有一個「奇怪值」 d_i 。只要 tw87 在地圖上記錄了該條道路，該道路的奇怪值就會累加到總和中。

tw87 想在滿足自己惡趣味的前提下，盡可能最小化總奇怪值，以免被可愛的學弟討厭。身為一中的彩虹新秀，請你幫 tw87 計算出最小可能的總奇怪值為多少？

— 輸入 —

第一行輸入二正整數 n m

接下來 m 行，輸入二正整數與一非負整數 a_i, b_i, d_i

— 輸出 —

輸出一個整數，代表最小可能的總奇怪值。

若無法構造出滿足條件的地圖，則輸出 -1。

— 輸入限制 —

- $2 \leq n \leq 2000$
- $1 \leq m \leq \min(10^4, n \times (n - 1))$
- $1 \leq a_i, b_i \leq n, a_i \neq b_i$
- $(a_i, b_i) \neq (a_j, b_j), 1 \leq i < j \leq m$
- $0 \leq d_i \leq 10^5$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$2 \leq n \leq 8, 1 \leq m \leq \min(30, n \times (n - 1)), 0 \leq d_i \leq 100$
3	17	$2 \leq n \leq 18, 1 \leq m \leq \min(200, n \times (n - 1)), 0 \leq d_i \leq 100$
4	22	$2 \leq n \leq 1000, 1 \leq m \leq \min(5000, n \times (n - 1)), 0 \leq d_i \leq 1$
5	24	$2 \leq n \leq 200, 1 \leq m \leq \min(1000, n \times (n - 1))$
6	29	無額外限制

－ 範例輸入 1 －

4 6
1 2 5
1 3 1
2 3 2
2 4 2
3 2 3
3 4 4

－ 範例輸出 1 －

6

－ 範例解釋 1 －

若選擇道路 $1 \rightarrow 3$ 、 $3 \rightarrow 2$ 、 $2 \rightarrow 4$ ，家豪會沿著 $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ 抵達開幕式，總奇怪值為 $1 + 3 + 2 = 6$ 。

另一種可行選法是 $1 \rightarrow 2$ 、 $2 \rightarrow 3$ 、 $3 \rightarrow 4$ ，總奇怪值為 $5 + 2 + 4 = 11$ ，因此最小可能總和為 6。

－ 範例輸入 2 －

3 2
1 2 4
1 3 1

－ 範例輸出 2 －

-1

－ 範例解釋 2 －

景點 2 不是開幕式景點，卻沒有任何從景點 2 出發的道路，無法替它選擇指引。因此無法構造符合條件的地圖，答案為 -1。

－ 範例輸入 3 －

5 8
1 4 2
4 5 2
2 3 1
3 2 1
1 2 10
2 4 10
3 5 10
4 3 10

－ 範例輸出 3 －

6

－ 範例解釋 3 －

可以選擇道路 $1 \rightarrow 4$ 、 $4 \rightarrow 5$ 作為主線，並讓其餘景點形成 $2 \rightarrow 3 \rightarrow 2$ 的循環，總奇怪值為 $2 + 2 + 1 + 1 = 6$ 。

任何合法地圖都必須選擇 $n - 1 = 4$ 條道路，而本例中四條最小的道路奇怪值總和已是 6，所以不可能得到更小的答案。

B. 小雞過馬路

Problem ID: Chicks_Go

Time Limit: 1s

Memory Limit: 512MiB



車輛來來去去的馬路上有一條斑馬線。

這條斑馬線的每個白色區塊（共 n 個）可以視為在一維數線上依序排列，位置依序處於 $x_1, x_2, \dots, x_n$ 。

今天有 m 隻小雞位於第一塊白色區塊（ x_1 ），想要到達最後一塊區塊（ x_n ）後，再折返回第一塊區塊。

然而，斑馬線上的白色區塊會因為小雞們的踩踏而磨損。除了第一塊與最後一塊區塊異常堅固（不會磨損）之外，對於中間的任何一個白色區塊 i ，只要有一隻小雞跳到上面，它的被踩踏次數就會加 1。當該區塊總共被踩踏了 a_i 次後，就會變成與路面一樣的顏色而消失（不再屬於斑馬線）。

註：所有跳躍依某個順序發生。第 a_i 次落在區塊 i 上仍然合法，該次落地後區塊消失，之後不得再有小雞落在該區塊上。

小雞們是很遵守交通規則的，所以牠們無論去程或回程，都只會踩在還沒消失的白色區塊上。假設每隻小雞的跳躍能力相同，請你幫忙計算：為了讓 m 隻小雞都能順利完成往返目標，小雞們的「單次最大跳躍距離」至少需要是多少？

— 輸入 —

第一行輸入兩個正整數 n m

第二行輸入 n 個整數 $x_1, x_2, \dots, x_n$

第三行輸入 $n - 2$ 個正整數 $a_2, a_3, \dots, a_{n-1}$

— 輸出 —

輸出小雞們的「單次最大跳躍距離」至少需要是多少。

— 輸入限制 —

- $3 \leq n \leq 10^5$
- $1 \leq m \leq 10^5$
- $|x_i| \leq 10^9$
- $x_1 < x_2 < \dots < x_n$
- $1 \leq a_i \leq 10^9$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	7	$m = 1, a_i = 10^9$
3	18	$3 \leq n \leq 2000, m = 1, a_i = 1$
4	9	$3 \leq n \leq 7, 1 \leq m \leq 7$
5	20	$3 \leq n \leq 100, 1 \leq m \leq 100, \sum_{i=2}^{n-1} a_i \leq 10^4$
6	22	$3 \leq n \leq 2000$
7	24	無額外限制

－ 範例輸入 1 －

```
3 1
0 1 2
1
```

－ 範例輸出 1 －

```
2
```

－ 範例解釋 1 －

若最大跳躍距離小於 2，小雞去程時必須踩到中間的白色區塊，使它在被踩踏 1 次後消失，回程便無法再使用它。

當最大跳躍距離為 2 時，小雞可以在去程與回程都直接往返第一塊與最後一塊白色區塊，因此答案為 2。

－ 範例輸入 2 －

```
5 2
-5 -1 2 3 10
3 1 5
```

－ 範例輸出 2 －

```
7
```

－ 範例輸入 3 －

```
6 3
-20 -19 -4 0 1 30
6 1 5 1000000000
```

－ 範例輸出 3 －

```
29
```

C. 一棵簡單的樹

Problem ID: Simple_Tree

Time Limit: 1s

Memory Limit: 512MiB

給定一棵以節點 1 為根、包含 N 個節點的樹。節點 i 上一開始有 A_i 顆石頭。

Alice 與 Bob 輪流行動，由 Alice 先手。每次行動可以選擇一個不是根、且仍有石頭的節點，從中拿走任意正整數顆石頭，並將這些石頭全部移到該節點的親代節點 (parent node)。無法行動的人輸掉遊戲。

— 輸入 —

第一行包含一個整數 N ，代表樹的節點數。

第二行包含 N 個整數 $A_1, A_2, \dots, A_N$ ，其中 A_i 代表節點 i 一開始的石頭數量。

接下來 $N - 1$ 行，每行包含兩個整數 u_i, v_i ，代表節點 u_i 與節點 v_i 之間有一條無向邊。

— 輸出 —

若 Alice 有必勝策略，輸出 Alice；否則輸出 Bob。

— 輸入限制 —

- $1 \leq N \leq 2 \times 10^5$
- $0 \leq A_i \leq 10^9$
- $1 \leq u_i, v_i \leq N$
- 輸入的 $N - 1$ 條邊構成一棵樹。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	9	$N \leq 2$
3	16	除了根節點外，每個節點的親代皆為節點 1
4	17	對所有 $1 \leq i < N$ ，第 i 條邊連接節點 i 與 $i + 1$
5	21	對所有 $1 \leq i \leq N$ ， $0 \leq A_i \leq 1$
6	37	無額外限制

— 範例輸入 1 —

2
1 1
1 2

— 範例輸出 1 —

Alice

— 範例輸入 2 —

3
0 5 5
1 2
1 3

— 範例輸出 2 —

Bob

－ 範例輸入 3 －

```
5
100 6 0 0 10
1 2
2 3
2 4
1 5
```

－ 範例輸出 3 －

Alice

－ 範例解釋 3 －

Alice 先將節點 5 上的 4 顆石頭移到節點 1，使節點 2 與節點 5 都剩下 6 顆石頭。此時其他非根節點都沒有石頭。

之後每當 Bob 從節點 2 或節點 5 移動若干顆石頭時，Alice 都能從另一個節點移動相同數量的石頭。最後 Bob 會先無法行動，因此 Alice 獲勝。

D. Merge, Rebase, Squash!

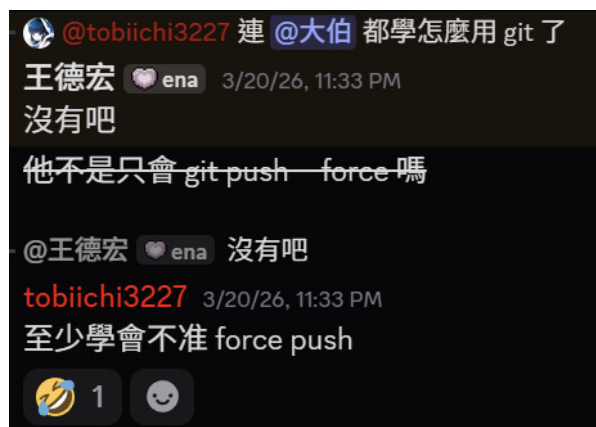
Problem ID: Git

Time Limit: 5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學大伯 force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者。TOJ 的程式碼由 H 個互相獨立的 hunk 組成，正式版本位於 branch v2.0。

王德宏身為 TOJ 開發團隊的成員之一，有天想為 TOJ 貢獻一個酷炫的功能，於是很高興地開啟了新的 branch feat/fancy，並在上面依序進行了 N 次操作。

王德宏想讓這項功能進到 TOJ 的 branch v2.0，於是請 tobiichi3227 進行審核。tobiichi3227 覺得 N 個 commits 太多了，應該先用 squash 將它們整理成恰好 K 個 squash commits，再依序 rebase 到 v2.0。

不同的 squash 方式可能產生不同的 conflict 成本。然而，王德宏要去參加國際地理奧林匹亞 (IGGO，全名 International GeoGuesser Olympiad)，已經沒有時間處理了，於是把最小化 conflict 成本的任務交給正在看題目的您。

不用擔心，下面會告訴您 branch rebase squash conflict 的定義。

完整流程合併動畫：

。

– Branch 與狀態 –

feat/fancy 一開始的所有 hunk 內容皆為 0。

令 $S_i[h]$ 表示執行完前 i 次操作後，第 h 個 hunk 的內容，因此 $S_0[h] = 0$ 。

第 i 次操作會把第 x_i 個 hunk 改成 v_i ，也就是：

$$S_i[h] = \begin{cases} v_i, & h = x_i, \\ S_{i-1}[h], & h \neq x_i. \end{cases}$$

另一方面，v2.0 上第 h 個 hunk 的內容為 U_h 。



Figure 2: 兩個 branches 從共同位置分開前進

完整動畫：

。

– Squash –

您需要將這 N 次操作依序切分成恰好 K 個連續的區間（即 K 個 squash commits）。

假設其中一個區間包含了第 L 到第 R 次操作（ $1 \leq L \leq R \leq N$ ），該區間對第 h 個 hunk 的實質修改定義為：

- 修改前（舊內容）： $\text{old} = S_{L-1}[h]$
- 修改後（新內容）： $\text{new} = S_R[h]$

若 $\text{old} = \text{new}$ ，則視為該區間對第 h 個 hunk 沒有進行修改。

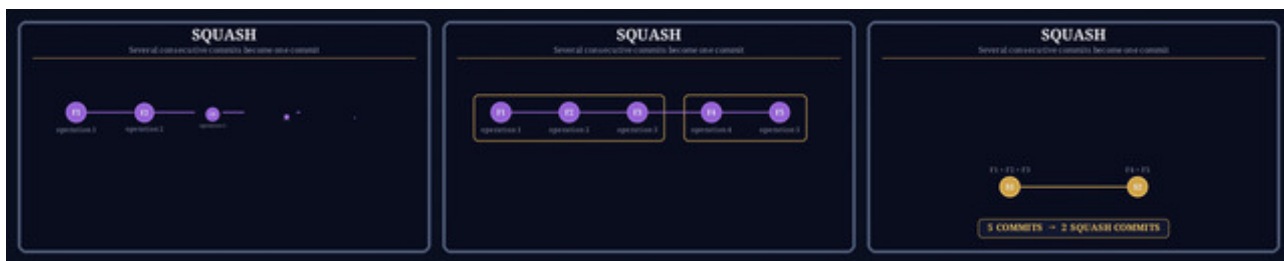


Figure 3: 每個連續區間形成一個 squash commit

完整動畫：

。

– Rebase –

這 K 個區間會按照原本順序逐一套用。

令 $\text{current}[h]$ 表示 v2.0 上第 h 個 hunk 目前的內容，一開始 $\text{current}[h] = U_h$

處理一個區間時，會分別考慮每個 hunk 的 old、new 與 $\text{current}[h]$ 。

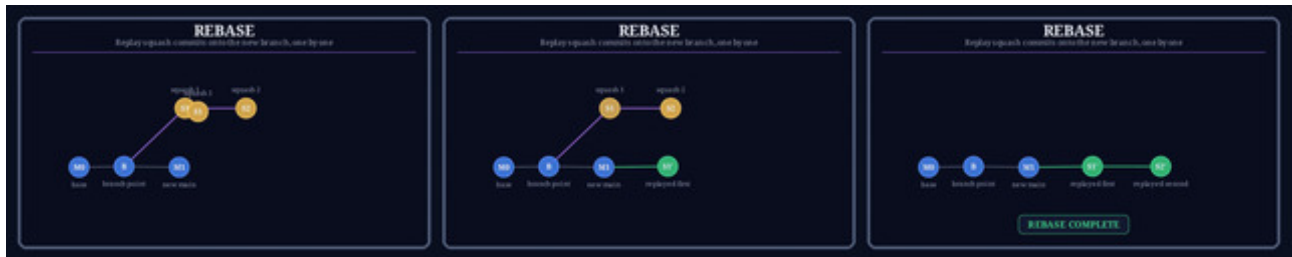


Figure 4: 各 squash commits 依序 replay 到 v2.0

完整動畫：

。

– Conflict –

若 $old = new$ ，該 hunk 沒有被區間修改， $current[h]$ 不變。

否則，只有當 $current[h] \neq old$ 且 $current[h] \neq new$ 時才會發生 conflict，並產生 w_h 的成本。

不論是否發生 conflict，處理完這項實質修改後皆令 $current[h] \leftarrow new$ 。

每個區間、每個 hunk 的 conflict 成本分別計算。



Figure 5: 發生 conflict 的兩個 commits 閃紅，解決後轉為綠色

完整動畫：[https://www.youtube.com/watch?v=...](#)。

— 輸入 —

第一行包含三個整數 N, H, K ，分別表示：

- feat/fancy branch 上的 commit 數量
- hunk 的數量
- 需要切分出的連續區間數量

第二行包含 H 個整數 U_h ($1 \leq h \leq H$)，表示 v2.0 branch 上第 h 個 hunk 的最終內容。

第三行包含 H 個整數 w_h ($1 \leq h \leq H$)，表示解決第 h 個 hunk 的 conflict 所需的成本。

接下來 N 行，第 i ($1 \leq i \leq N$) 行包含兩個整數 x_i 與 v_i ，表示第 i 個 commit 將第 x_i 個 hunk 的內容改成 v_i 。

— 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

— 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit i ： $v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且不會將任何 hunk 改回 0。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	3	$N, H \leq 15$
3	7	$K = 1$
4	28	每個 hunk 最多只被修改一次
5	13	$N \leq 1000$
6	49	無限制

— 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

— 範例輸出 —

```
0
```

— 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

$(0, 0) \rightarrow (1, 0) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 4) \rightarrow (3, 4)$

可以將 commits 分成 (1), (2, 3, 4, 5)。

第一個 squash commit 將 h_1 從 0 改成 1，套用到 v2.0 branch 時， h_1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的將 h_1 從 1 改成 3，將 h_2 從 0 改成 4，此時 h_1 的目前內容為 1，可以正常套用， h_2 的目前內容已經是 4，因此也不會發生 conflict，總成本為 0。

— 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

— 範例輸出 —

```
7
```

— 範例說明 —

因為 $K = 1$ ，兩個 commits 必須全部 squash 在一起，squash 後的效果為將 0 改成 2。

但 v2.0 branch 上該 hunk 的內容為 1，目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，總成本為 7。

— 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

— 範例輸出 —

```
0
```

— 範例說明 —

因為 $K = 2$ ，兩個 commits 分別套用。

第一個 commit 將 0 改成 1 而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。

第二個 commit 將 1 改成 2，此時目前內容為 1，可以正常套用。

因此總成本為 0。

– Git 小教室 –

– 下面內容不閱讀不影響解題 –

這裡用一個小型 repository，把題目裡的 branch、squash、rebase 和 conflict 真的實現一次。

不需要網路，toj.git 是放在同一台電腦上的本機 remote，不會連到 GitHub。

以下命令以 Bash 或 Git Bash 為例，假設大家會基本的 Linux CLI。

Git 到底在做什麼？

先解釋後面會出的名詞

- **working tree**：現在資料夾裡看到的檔案。
- **staging area**：下一個 commit 要收進去的內容。
- **commit**：某個時間點的檔案狀態，以及它的前一個 commit。
- **branch**：指向某個 commit 的名稱。建立新 commit 後，branch 會跟著往前走，例如 v2.0、feat/fancy。
- **HEAD**：您現在所在的位置，通常是某個 branch。
- **remote**：另一個 repository 的名稱。它可以在網路上，也可以只是本機的一個路徑。

平常用到的命令大概是這些：

<code>git init</code>	# 建立 <i>repository</i>
<code>git status</code>	# 查看目前 <i>branch</i> 、已修改與已 <i>stage</i> 的檔案
<code>git diff</code>	# 查看尚未 <i>stage</i> 的差異
<code>git diff --staged</code>	# 查看已 <i>stage</i> 、將進入下一個 <i>commit</i> 的差異
<code>git add <file></code>	# 將檔案的目前內容放進 staging area
<code>git commit -m "<message>"</code>	# 建立 <i>commit</i>
<code>git log --graph --oneline --all</code>	# 用簡圖查看所有 <i>branch</i> 的歷史
<code>git branch</code>	# 列出 <i>branch</i>
<code>git switch <branch></code>	# 切換 <i>branch</i>
<code>git switch -c <branch></code>	# 建立並切換到新 <i>branch</i>
<code>git merge <branch></code>	# 把指定 <i>branch</i> 合併到目前 <i>branch</i>
<code>git rebase <branch></code>	# 把目前 <i>branch</i> 的 <i>commits</i> 逐一重播到指定 <i>branch</i>

`git commit -am "<message>"` 可以看成 `git add` 已追蹤檔案再 `git commit` 的縮寫。
不過，新建立跟還沒被 Git 追蹤的檔案不會被加進去。

不確定現在 repository 是什麼狀態時，先執行：`git status`

1. 建立完全離線的 repository

先建立一個新的資料夾。toj.git 當作遠端的 TOJ repository，work 則是平常工作的 repository：

```
mkdir git-classroom
cd git-classroom
git init --bare toj.git
git init -b v2.0 work
cd work
```

bare repository 沒有 working tree，所以適合拿來當 remote。

接著只設定這次教學使用的作者資料，email 隨便填就好。

```
git config user.name "Git Student"
git config user.email "student@example.com"
```

接下來用兩個一行文字檔模擬兩個 hunk，先建立共同的起點：

```
printf "0\n" > hunk-1.txt
printf "0\n" > hunk-2.txt
git add hunk-1.txt hunk-2.txt
git status
git commit -m "base: all hunks are 0"
git tag base
```

base tag 只是幫這個 commit 取一個固定名字。後面做 squash 時，可以直接寫 base，不需要一直複製 commit hash。

2. 在 feat/fancy 建立四個 commits

從共同的起點開一個 feature branch 並切換過去：

```
git switch -c feat/fancy
```

接著改幾次檔案，每次都做一次 commit。這裡故意讓同一個 hunk 被修改不只一次：

```
printf "3\n" > hunk-1.txt
git commit -am "fancy 1: hunk 1, 0 to 3"

printf "4\n" > hunk-2.txt
git commit -am "fancy 2: hunk 2, 0 to 4"

printf "5\n" > hunk-1.txt
git commit -am "fancy 3: hunk 1, 3 to 5"

printf "7\n" > hunk-1.txt
git commit -am "fancy 4: hunk 1, 5 to 7"
```

現在可以 git log 看這四個 feature commits：

```
git log --oneline --decorate
```

把本機的 bare repository 設成 origin，再把目前的 branch 推上去：

```
git remote add origin ../toj.git
git push -u origin feat/fancy
```

-u 會記住本機 feat/fancy 對應的是 origin/feat/fancy。

這裡的 remote 是 ../toj.git，所以整個 push 都是在本機操作。

3. 讓 v2.0 也往前走

切回 v2.0，把兩個 hunk 設成最後的 (5, 4)：

```
git switch v2.0
printf "5\n" > hunk-1.txt
printf "4\n" > hunk-2.txt
git commit -am "v2.0: set final hunk states"
git push -u origin v2.0
git log --graph --oneline --decorate --all
```

現在兩條 branch 從 base 分開了：

```
          fancy 1 -- fancy 2 -- fancy 3 -- fancy 4  (feat/fancy)
        /
base: (0, 0) --
        \
          v2.0: (5, 4)                                (v2.0)
```

4. Merge 和 Rebase 差在哪裡？

先看 merge。

這裡另外開一條 merge-demo，不影響後面真正要做的 rebase：

```
git switch -c merge-demo v2.0
git merge feat/fancy
```

這時 Git 會遇到 conflict。

原因很簡單，因為兩邊都改了 hunk-1.txt，但改成的內容不同。

hunk-2.txt 就沒有這個問題。兩邊最後都是 4，所以 Git 可以自己處理。

可以用下面的命令看目前狀態：

```
git status
git diff
```

這次只是拿 merge 來比較，所以不需要留下這個結果。放棄 merge，回到 feature branch：

```
git merge --abort
git switch feat/fancy
```

rebase 的做法不一樣。

執行：

```
git rebase v2.0
```

Git 會先找到 feat/fancy 和 v2.0 的共同祖先 (LCA)，接著把 feature branch 上的 commits 暫時拿下來，再一個一個套到新版的 v2.0 後面，所以不會出現像 cerge 一樣的 merge commit。

有一點要注意：rebase 不是直接把 branch 名字搬過去，commit 重新建立後，parent 和內容雜湊可能都會改，所以 commit hash 也會跟著變。

5. 把四個 commits squash 成兩個

題目要求把連續的 commits 分組。

這裡分成 [1,2] [3,4]，所以最後會剩兩個 commits，也就是 $K = 2$ 。

目前在 feat/fancy 上，執行：

```
git rebase -i base
```

編輯器會列出四個 commits，順序是由舊到新。hash 每台電腦都可能不同，所以只看排列就好：

```
pick <hash 1> fancy 1: hunk 1, 0 to 3
pick <hash 2> fancy 2: hunk 2, 0 to 4
pick <hash 3> fancy 3: hunk 1, 3 to 5
pick <hash 4> fancy 4: hunk 1, 5 to 7
```

每一組裡，第一個維持 pick，後面的改成 squash：

```
pick    <hash 1> fancy 1: hunk 1, 0 to 3
squash  <hash 2> fancy 2: hunk 2, 0 to 4
pick    <hash 3> fancy 3: hunk 1, 3 to 5
squash  <hash 4> fancy 4: hunk 1, 5 to 7
```

儲存並關閉編輯器。

如果 Git 接著要求修改 squash 後的 commit message，再確認內容、儲存並關閉即可。

第一個 squash commit 的內容等於：

```
hunk 1: 0 -> 3
hunk 2: 0 -> 4
```

第二個則是：

```
hunk 1: 3 -> 7
```

也就是原本的 3 -> 5 -> 7 被合成一次 3 -> 7。

可以確認現在 base 後面只剩兩個 commits：

```
git rev-list --count base..feat/fancy
git log --oneline --reverse base..feat/fancy
```

第一個命令應該輸出：2

如果是真實 repository，沒有事先建立 base tag，也可以先找共同祖先：

```
git merge-base v2.0 feat/fancy
```

把輸出的 commit hash 拿去做：

```
git rebase -i <hash>
```

6. Rebase，然後處理 conflict

現在把這兩個 squash commits 套到 v2.0：

```
git rebase v2.0
```

第一個 squash commit 套上去時，hunk-1.txt 會衝突。

原因是：

squash commit 想把 hunk-1 從 0 改成 3。但 v2.0 上現在已經是 5。

所以 Git 不知道這裡應該怎麼套。

hunk-2.txt 則沒有 conflict，因為 squash commit 想要的結果是 4，而 v2.0 目前也是 4。

先看狀態：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

hunk-1.txt 會看到 Git 放進去的 conflict markers，大概長這樣：

```
目前版本的起點標記 | <<<<<< HEAD
目前版本的內容      | 5
分隔線              | =====
正在 replay 的內容   | 3
feature 版本的標記   | >>>>>> fancy 1: hunk 1, 0 to 3
```

左邊那些說明文字只是為了方便閱讀，實際檔案裡只有右邊的內容。

<<<<<<、=====、>>>>>> 都是 Git 暫時加進去的標記，方便解決衝突（但我覺得這個也很抽象）。

題目規定這次 conflict 要採用 squash commit 的 new，所以最後把 hunk-1.txt 改成 3：

```
printf "3\n" > hunk-1.txt
git add hunk-1.txt
git rebase --continue
```

這裡的 git add 不只是「準備 commit」，也代表您已經告訴 Git：這個 conflict 處理好了。

如果 git rebase --continue 開啟 commit message 編輯器，確認後儲存並關閉即可。

接下來第二個 squash commit 是 3 -> 7，這次可以直接套用，rebase 應該會完成。

最後檢查：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

```
git rev-list --count v2.0..feat/fancy
git log --graph --oneline --decorate v2.0 feat/fancy
```

最後應該是：

```
hunk-1.txt = 7
hunk-2.txt = 4
```

而 v2.0 後面有兩個 feature commits。

如果 rebase 做到一半發現不想繼續，可以：`git rebase --abort`

它會把這次 rebase 放棄，回到開始之前的狀態。

至於 `git rebase --skip` 要小心。它不是「跳過這個 conflict 再繼續」，而是直接放棄目前正在 replay 的那個 commit。不了解狀況時不要隨使用。

7. 為什麼 rebase 後要 force push？

這時 remote 還留著原本那四個 commits，但本機的 feat/fancy 已經變成另外兩個 commits。

普通的 push 不會接受這種歷史改寫，所以：

```
git push origin feat/fancy
```

應該會被拒絕。

如果確認 remote 沒有人偷偷加上新的 commits，可以使用：

```
git push --force-with-lease origin feat/fancy
```

`--force-with-lease` 和單純的 `--force` 不太一樣。

它會先確認 remote branch 還是您預期的狀態。如果有人在這段時間更新過 remote，push 就會被拒絕，而不是直接把對方的 commits 蓋掉。

不過這仍然是在改寫 remote 歷史。多人共用的 branch 在這麼做之前，還是要先確認其他人沒有正在使用它。

題目模型和現實 Git 的差異

題目裡可以直接用 old、new、current 三個整數來判斷某個 hunk 有沒有 conflict。現實的 Git 沒這麼單純，它還會看檔案中的多行 context，也會處理檔案新增、刪除、重新命名，以及彼此靠得很近的修改。

這次教學把兩個 hunk 各自放在一個一行文字檔裡，所以可以把題目中的情況跑出來。

E. 你可以幫我吹口哨嗎?

Problem ID: Can_You_Blow_My_Whistle

Time Limit: 1s

Memory Limit: 512MiB



Figure 1: Can You Blow My Whistle Baby

本題是互動題型。

國中營比賽當天，中午吃飯的時候，有 n 個小學弟排成一列準備領便當。

小學弟們身上穿著 1 到 n 號的衣服，且每個號碼恰好出現一次。

目前他們站在位置 1 到 n 上，位於位置 i 的學弟是 a_i 。

身為大學長的你，希望他們能依照號碼 1 到 n 的順序從小到大排好（也就是位置 i 要站的是編號 i 的學弟）。

為了節省時間，你決定吹哨子發號施令。每次吹哨，你可以同時指定任意多組學弟互換位置，但為了避免碰撞，每個學弟在同一次吹哨中，最多只能參與一組交換。

身為口哨先輩的你，請安排交換方式，用盡可能少的哨音讓所有學弟排好。

— 實作細節 —

你需要完成以下函式：

- `void solve(int n, std::vector<int> a);`
 - a 的長度為 n ，且是 1 到 n 的一個排列。
 - 對於 $0 \leq i < n$ ， $a[i]$ 表示一開始站在位置 $i + 1$ 的學弟編號。

上述程序的執行過程你需要呼叫以下函式：

- `void swap_student(int u, int v);`
 - $1 \leq u, v \leq n$ 且 $u \neq v$ 。
 - 將「交換位置 u 與位置 v 」加入目前這一輪，但此時還不會立刻交換。
 - 同一輪中，每個位置最多只能出現在一次 `swap_student` 呼叫中。
- `void blow_whistle();`
 - 同時執行目前這一輪加入的所有交換，接著開始新的一輪。
 - 即使這一輪沒有加入任何交換，呼叫此函式仍會計入一次哨音。

傳入的 a 是普通的 `std::vector<int>`；呼叫上述函式不會自動修改你程式中的 a 。
若 `solve` 結束時仍有尚未經過 `blow_whistle` 執行的交換，這些交換不會生效。

— 條件限制 —

- $1 \leq n \leq 10^5$
- $1 \leq a_i \leq n$
- a 是一個 $1 \sim n$ 的排列

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	34	$1 \leq n \leq 2000$
3	15	保證最佳哨音次數為 1
4	51	無額外限制

— 給分方式 —

假如你呼叫 `blow_whistle` 函式的次數超過了 $2n$ 次，則系統會中斷，並給出 **Wrong Answer**。

當你的 `solve` 函式結束時，系統會檢查學弟們是否已經正確排序。若最終沒有完成排序，該筆測資的得分倍率為 0。

若成功完成排序，令該筆測資理論上最少需要的哨音數為 opt ，你呼叫 `blow_whistle` 的總次數為 A ，該筆測資的得分倍率 T 為：

- 若 $A = opt$ ，則 $T = 1$ 。
- 若 $A > opt$ ，則 $T = \max(0, 0.5 - 0.02(A - opt))$ 。

也就是說，只要哨音數不是最佳解，該筆測資的倍率最多為 0.5，且每多吹一次會再扣除 0.02，直到降為 0。

對於滿分為 P 的子任務，令 $T_{\min}$ 為你在該子任務所有測資中得到的最小倍率，則此子任務的得分為 $P \times T_{\min}$ 。

— 範例 —

考慮以下呼叫。

```
solve(5, {2, 1, 3, 5, 4})
```

你可以在 `solve` 函式中依序呼叫：

```
swap_student(1, 2);  
swap_student(4, 5);  
blow_whistle();
```

前兩個呼叫將兩組交換加入同一輪；呼叫 `blow_whistle` 後，兩組交換同時執行，排列變成 $[1, 2, 3, 4, 5]$ 。此例最少需要一次哨音。

— 範例評分程式 —

輸入格式：

$$n$$
$$a_0, a_1, \dots, a_{n-1}$$

若呼叫函式過程中不違法，則輸出格式如下：

$$A : \text{吹哨次數}$$
$$a'_0, a'_1, \dots, a'_{n-1}$$

否則，格式為：

Wrong Answer: MSG

其中 MSG 是錯誤的原因，包含以下結果：

- `invalid swap`：你呼叫 `swap_student` 的 `u`, `v` 是不合法的。
- `twice swap`：有一個位置在同一輪交換中被 `swap_student` 函式呼叫到兩次以上了。

範例評分程式會在發現以上錯誤時回報錯誤並立即結束程式。除了以上有提到的錯誤之外，範例評分程式**不會**另外檢查答案的正確性。

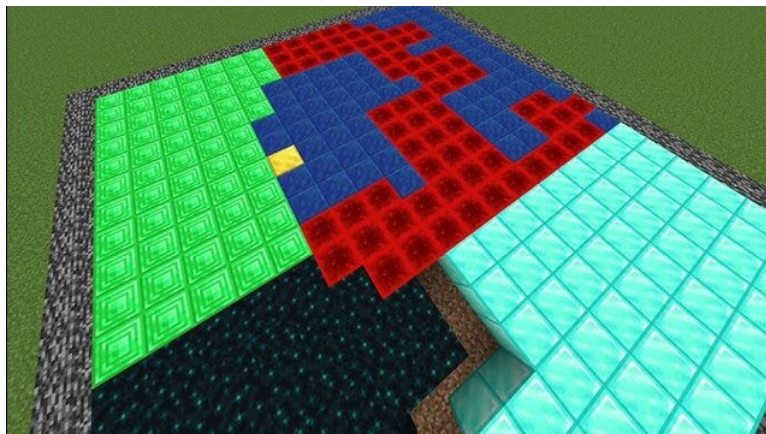
正式的評分程式不一定採用以上程式輸入。請不要自行處理輸入輸出。

F. 礦石戰爭

Problem ID: Redstone_NOOOOO

Time Limit: 1s

Memory Limit: 512MiB



起初，世人總以為「青金石國與紅石國的關係特別好」是句玩笑話…

已知，在礦石大陸上有著 N 個國家，編號為 $1 \sim N$ 。

每個國家位於 A_i 的地理位置，且每個國家皆具有國庫寶藏收藏值 B_i ，只是當戰爭還未打響時，每個國家的 B_i 皆為 0。

戰爭打響了，在戰火的號角下，礦石大陸發生了史無前例的大戰。而戰爭可以用 Q 次事件描述。

對於每次事件，總共有 5 種類型：

- **結盟** 1 a b ：若 a 國與 b 國不處於同個同盟國，則將 a 與 b 兩國的同盟國合併，否則視為操作無效。
- **背刺** 2 a b ：若 a 國與 b 國不處於同個同盟國，則 a 國脫離原先的同盟國，並加入進 b 國的同盟國中，否則操作無效。
- **寶藏出世** 3 l r x ：對於地理位置 A_i 處於 $[l, r]$ 的國家， B_i 皆增加 x ，若無國家處於 $[l, r]$ 地理位置中，或 $x = 0$ ，則視此操作無效。
- **詢問** 4 a ：詢問 a 國所在同盟國中，所有國家 B_i 的總和。
- **時空逆轉** 5：對於最近一個 $1 \sim 3$ 的有效操作，進行回溯，使得回到該個操作發生變化之前，可連續回溯，已經回溯的操作不會再次成為回溯對象。

— 輸入 —

第一行包含兩個整數 N, Q ，分別代表國家的數量與操作數量。

第二行包含 N 個整數 $A_1, A_2, \dots, A_N$ 。

接下來 Q 行，每行為下列其中一種格式：

- 1 a b
- 2 a b
- 3 l r x
- 4 a
- 5

— 輸出 —

對每個第 4 種操作，依序輸出一行，包含一個整數，代表該同盟國中所有國家的 B_i 總和。

— 輸入限制 —

- $1 \leq N, Q \leq 5 \times 10^4$
- $0 \leq A_i \leq 10^9$
- $1 \leq a, b \leq N$
- $0 \leq l \leq r \leq 10^9$
- $-10^9 \leq x \leq 10^9$
- 每次執行操作 5 時，至少存在一個尚未被撤銷的有效操作。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	13	$N, Q \leq 2000$
3	11	只出現第 3, 4, 5 類操作
4	9	$A_1 = A_2 = \dots = A_N$
5	17	A_i 中不同的值不超過 32 種
6	19	不出現第 2 類操作
7	31	無額外限制

— 範例輸入 1 —

```
1 5
5
4 1
3 5 5 -7
4 1
5
4 1
```

— 範例輸出 1 —

```
0
-7
0
```

— 範例說明 1 —

第一次詢問時，國家 1 的 $B_1 = 0$ 。接著操作 3 讓 B_1 增加 -7 ，所以第二次詢問得到 -7 。最後操作 5 撤銷這次的寶藏修改，使 B_1 恢復為 0 。

－ 範例輸入 2 －

```
4 11
1 5 5 10
3 5 10 4
1 1 2
4 1
2 2 3
4 3
5
4 3
5
4 1
5
4 2
```

－ 範例輸出 2 －

```
4
8
4
0
0
```

－ 範例說明 2 －

第一次操作後，各國寶藏值 $B = (0, 4, 4, 4)$ 。將國家 1, 2 所在的同盟國合併後，詢問國家 1 得到 4。接著把國家 2 移到國家 3 所在的同盟國（背刺），該同盟國的總和為 $4 + 4 = 8$ 。

第一次撤銷會還原背刺（移動）操作，所以詢問國家 3 得到 4；第二次撤銷會還原結盟（合併）操作，所以詢問國家 1 得到 0；第三次撤銷會還原最初的寶藏出世（區間修改），所以詢問國家 2 也得到 0。

－ 範例輸入 3 －

```
3 13
2 4 6
1 1 2
1 1 2
3 3 3 5
3 2 6 0
4 1
5
3 2 2 7
2 1 2
4 2
5
4 2
5
4 1
```

－ 範例輸出 3 －

```
0
7
0
0
```

－ 範例說明 3 －

第一次結盟國家 1, 2 的操作成功，但緊接著再次結盟同一對國家不會成功。後面的兩次寶藏出世也分別因為區間內沒有符合的國家與 $x = 0$ 而不會成功；詢問操作同樣不會進入回溯紀錄。因此第一次撤銷會直接撤銷最初那次成功的結盟。

接著對 $A_i = 2$ 的國家增加 7，再把國家 1 背刺移入國家 2 所在的同盟國，詢問得到 7。撤銷背刺移動後，國家 2 自己的 B 值為 0；再撤銷寶藏出世後，國家 1 的 B 值也恢復為 0。

G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

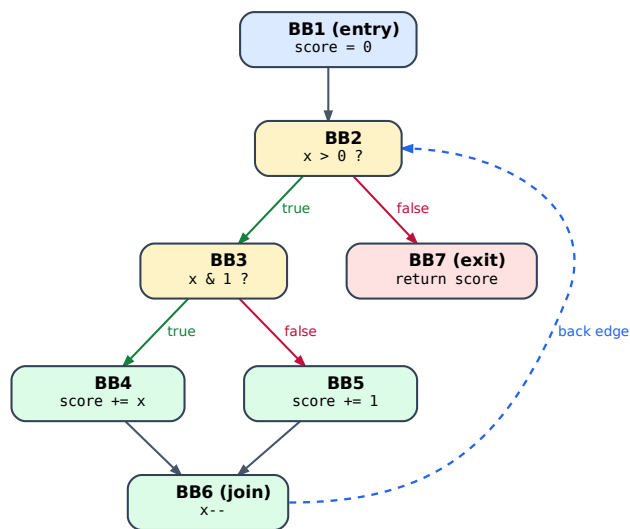


Figure 1: 一個含迴圈與條件分支的 CFG。每個框是一個 basic block； BB_6 是兩條分支的 join point， $BB_6 \rightarrow BB_2$ 是迴圈的 back edge。

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 在程式執行期間總共被通過幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有 N 個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從 BB_i 可以走到 BB_j ，總共有 M 條有向邊。

程式每執行一次，控制流都會從入口 1 開始，沿著 CFG 上的有向邊前進，最後抵達出口 N 結束執行。過程中每沿著一條有向邊走過一次，該邊的通過次數就增加 1；若因為迴圈而在同一次執行中重複通過同一條邊，則每次通過都會分別計數。程式總共執行 R 次，令 f_e 表示這 R 次執行合計通過邊 e 的次數。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 (BB_i 或 BB_j 都可以，編譯器只會選一個 BB 插入)，因為測量通過次數會造成額外的運算，所以插入每個計數器需要成本 c_e ，執行結束後就能知道該邊通過的次數。

每次控制流進入一個普通 BB 後，都一定會再沿著某條邊離開。因此，普通 BB 被進入的總次數會等於它被離開的總次數。入口 1 是每次執行的起點，所以離開次數會比進入次數多 R ；出口 N 則正好相反。也就是說：

- 對於入口與出口以外的每個 BB，所有連進該 BB 的邊，其通過次數總和，等於所有從該 BB 連出去的邊，其通過次數總和。

- 對於入口 1 ，所有連出去的邊的通過次數總和，等於所有連進來的邊的通過次數總和加上 R 。
- 對於出口 N ，所有連進來的邊的通過次數總和，等於所有連出去的邊的通過次數總和加上 R 。

利用這些進出次數的關係，可以從已知的邊通過次數推導出部分沒有安裝計數器的邊。

編譯器必須在程式執行前決定安裝計數器的邊。無論這 R 次執行分別採取哪些合法路徑，執行結束後都必須能由 R 、CFG、進出次數守恆關係與所有計數器的讀值，唯一確定每條邊的通過次數。求最小安裝成本，使得執行結束後能夠**唯一確定每一條邊通過的次數**。

— 輸入 —

第一行會有三個整數 N, M, R ，代表 BB 的數量、邊的數量與執行次數

接下來 M 行會有三個整數 u, v, C ，代表 BB_u 可以走到 BB_v ，且安裝計數器成本為 C

— 輸出 —

最小安裝成本

— 輸入限制 —

- $2 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到 N 的路徑通過

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	3	$M = N - 1$, 且第 i 條邊為 $i \rightarrow i + 1$
3	7	$C_e = 1$
4	13	$M \leq 20$
5	19	$M = N$ 且無 self-loop 或重複連接
6	21	每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2
7	37	無額外限制

— 範例輸入 —

4 4 8
1 2 5
1 3 2
2 4 4
3 4 6

— 範例輸出 —

2

— 範例解釋 —

輸入的 CFG 的結構如下：

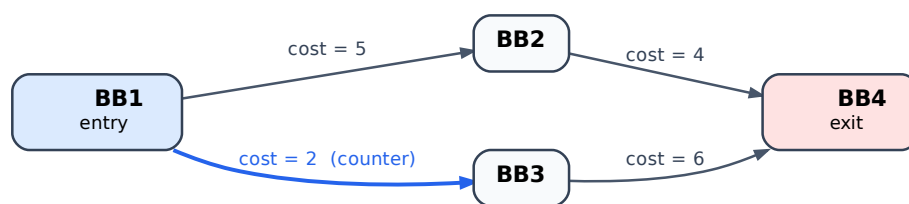


Figure 2: 範例一的 CFG。邊上標示安裝計數器的成本；藍色粗邊表示在 $1 \rightarrow 3$ 安裝計數器。

程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$ 或 $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊 $1 \rightarrow 3$ 上安裝計數器。

假設程式執行完畢後，計數器顯示邊 $1 \rightarrow 3$ 總共被通過 3 次。因為程式總共執行 8 次，所以邊 $1 \rightarrow 2$ 必定被通過 $8 - 3 = 5$ 次。

每次進入 BB_2 後都會沿著邊 $2 \rightarrow 4$ 離開，因此邊 $2 \rightarrow 4$ 也被通過 5 次。

同理，邊 $3 \rightarrow 4$ 被通過 3 次。

因此，只要在邊 $1 \rightarrow 3$ 上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過 BB_2 或 BB_3 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

— 範例輸入 —

4 7 14
1 2 8
2 4 5
1 3 7

3 4 4

2 3 6

2 2 3

1 2 9

— 範例輸出 —

21

— 範例輸入 —

6 6 12

1 2 9

2 3 8

3 4 3

2 4 7

4 5 6

5 6 5

— 範例輸出 —

3

— 範例輸入 —

8 9 18

1 2 10

2 3 8

3 5 2

2 4 6

4 5 5

5 6 7

6 7 4

7 8 3

6 8 9

— 範例輸出 —

5

－ 編譯器小教室：從 C Code 到 CFG －

C 的 if、while、for 很適合人看，但對編譯器來說，這些語法還是太有結構了。做最佳化時，通常會先把它們拆成幾種比較單純的控制流操作：

```
goto L
if condition goto L_true else goto L_false
return value
```

把原本的結構化控制流拆成 label 和 jump，通常叫做 **lowering**。

下面使用的 goto-style 是教學用的 pseudo IR。它和 LLVM IR 一樣，會用 label 和明確的 branch 表示控制流，但它不是合法的 **LLVM IR**。真正的 LLVM IR 還有型別、SSA、phi 等規則，而且 terminator 使用的是 br、ret、switch 等指令。

兩者可以先這樣理解：

```
goto L
    <=> br label %L

if c goto T else goto F
    <=> br i1 %c, label %T, label %F

return x
    <=> ret i32 %x
```

把大括號裡的 if、迴圈等結構拆成 label 和 goto，概念上就是 TOJ 756 在做的事情。

Basic Block (BB)

Basic block 是一段**最大且連續**的指令序列。它有幾個基本條件：

- 控制流只能從 block 的第一條指令進入，不能跳到中間
- block 執行到最後之前，不會跳到其他地方。
- 一旦進入 block，就會依序執行裡面的指令，直到遇到 terminator。

在 LLVM-like IR 中，每個 BB 從 label 開始，最後以一條 terminator 結束。terminator 會決定下一個 BB，或者直接結束函式。

如果現在手上只有一串 goto-style 指令，可以先找 **leaders**：

1. 函式的第一條指令是 leader。
2. 每個 jump 的目標 label 是 leader。
3. if、goto、return 後面如果還有指令，那條下一條指令也是 leader。

找到 leaders 之後，從一個 leader 開始，一直到下一個 leader 前的指令，都屬於同一個 BB。

實作時通常還會再處理 unreachable blocks，並視情況合併不必要的空 block。

從 BB 建立 CFG

CFG 的每個節點就是一個 BB。

如果執行完 BB_u 後，下一步可能執行 BB_v ，就在圖上加一條有向邊 $u \rightarrow v$ 。

- goto BB2：加入一條邊到 BB_2 。
- if c goto BB2 else BB3：加入 BB_2 和 BB_3 兩條邊。
- return：沒有函式內的 successor。
- 如果 IR 允許 fall-through，就把邊接到文字順序的下一個 BB。

本文會把 jump 都直接寫出來，所以不考慮隱藏的 fall-through。

也就是說，前面兩個步驟可以分開看：label 決定 BB 怎麼切，terminator 決定 CFG 的邊。

常見 C 控制結構如何 Lower ?

下面先把 C 程式改成 goto-style，再從 terminator 直接看 CFG。

1. 直線程式

```
x = a + b;  
y = x * 2;  
return y;
```

沒有 branch，所以整段就是一個 BB：

```
BB1:  
    x = a + b;  
    y = x * 2;  
    return y;
```

return 是 terminator。執行到這裡函式就結束了，因此後面不能再接其他可執行指令。

2. if / else

```
if (x < 0)  
    y = -x;  
else  
    y = x;  
use(y);
```

Lowering 後可以寫成：

```
BB1: if x < 0 goto BB2 else goto BB3  
BB2: y = -x; goto BB4  
BB3: y = x; goto BB4  
BB4: use(y); ...
```

所以 CFG 有 $1 \rightarrow 2$ 、 $1 \rightarrow 3$ 、 $2 \rightarrow 4$ 、 $3 \rightarrow 4$ 。

BB_4 是兩條分支重新接上的地方，也就是 join point。

如果原始程式只有 if 沒有 else，false edge 就可以直接接到後面的 join point。

3. while

```
while (x > 0) {  
    sum += x;  
    --x;  
}  
use(sum);
```

可以拆成：

BB1:

```
goto BB2
```

BB2:

```
if x > 0 goto BB3 else goto BB4
```

BB3:

```
sum += x;  
--x;  
goto BB2
```

BB4:

```
use(sum);  
...
```

BB_2 是 loop header。每次進入下一輪之前，都會在這裡重新檢查條件。

$BB_3 \rightarrow BB_2$ 形成 cycle； $BB_2 \rightarrow BB_4$ 則是離開迴圈的 exit edge。

4. do / while

```
do {  
    read_next();  
} while (!ok);  
use_data();
```

Lowering 後：

BB1:

```
    read_next();  
    goto BB2
```

BB2:

```
    if !ok goto BB1 else goto BB3
```

BB3:

```
    use_data();  
    ...
```

這裡條件檢查在 body 後面，所以第一次一定會先執行 `read_next()`。

這就是它和 while 在 CFG 上最明顯的差別。

5. for、break 與 continue

```
for (int i = 0; i < n; ++i) {  
    if (a[i] < 0) continue;  
    if (a[i] == 0) break;  
    sum += a[i];  
}  
use(sum);
```

for (init; cond; step) body 可以拆成 init、condition、body、step 和 exit：

```
BB1: i = 0; goto BB2  
BB2: if i < n      goto BB3 else goto BB7  
BB3: if a[i] < 0   goto BB6 else goto BB4  
BB4: if a[i] == 0  goto BB7 else goto BB5  
BB5: sum += a[i]; goto BB6  
BB6: ++i; goto BB2  
BB7: use(sum); ...
```

這裡很容易寫錯 continue。

continue 應該跳到 step 的 BB_6 ，不能直接跳回 condition。否則 $++i$ 不會執行。

break 則直接跳到 loop exit BB_7 。

6. Short-circuit : && 與 ||

```
if (a != 0 && b / a > 2)
    hit();
next();
```

&& 的右邊只有在左邊為 true 時才會計算，因此兩個條件會落在不同的 BB：

BB1:

```
if a != 0 goto BB2 else goto BB4
```

BB2:

```
if b / a > 2 goto BB3 else goto BB4
```

BB3:

```
hit();
goto BB4
```

BB4:

```
next():
...
```

所以當 $a == 0$ 時，控制流直接從 BB_1 到 BB_4 ，不會執行 b / a 。

$a || b$ 則剛好反過來。左邊如果已經是 true，就不需要再算右邊；只有左邊為 false 時才會繼續。

7. switch

```
switch (op) {  
  case 0: y = 10; break;  
  case 1: y = 20; break;  
  default: y = -1;  
}  
use(y);
```

最簡單的 lowering 可以寫成一串比較：

```
BB1: if op == 0 goto BB3 else goto BB2  
BB2: if op == 1 goto BB4 else goto BB5  
BB3: y = 10; goto BB6  
BB4: y = 20; goto BB6  
BB5: y = -1; goto BB6  
BB6: use(y); ...
```

實際的編譯器不一定真的產生這麼多比較。也可能保留多路的 switch terminator，或者進一步做成 jump table。

但不管最後的機器碼長什麼樣，CFG 上的 dispatch block 都可能有多個 successors，各個 case 最後可以再接回共同的 join point。

8. 提早 return

```
BB_return_negative:  
  result = -1;  
  goto BB_exit  
  
BB_return_answer:  
  result = answer;  
  goto BB_exit  
  
BB_exit:  
  return result;
```

每個 return 都會結束目前的 BB，所以在函式內沒有 successor。

如果某個 compiler pass 希望函式只有一個出口，可以把回傳值先存起來，再全部跳到同一個出口：

這種做法常見於 CFG normalization，但不是所有 compiler 都需要這樣處理。

完整範例：從 C、goto-style 到 CFG

```

int calc(int x) {
    int score = 0;
    while (x > 0) {
        if (x & 1)
            score += x;
        else
            score += 1;
        --x;
    }
    return score;
}

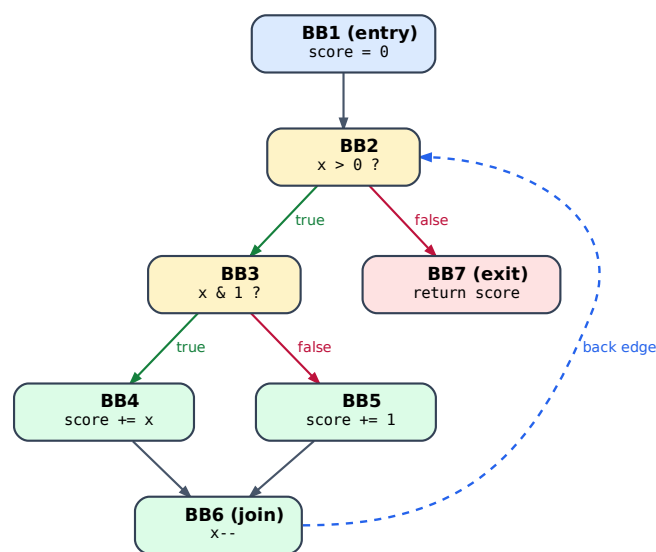
```

先把它改成只有明確 jump 的 pseudo IR：

```

BB1: score = 0; goto BB2
BB2: if x > 0 goto BB3 else goto BB7
BB3: if x & 1 goto BB4 else goto BB5
BB4: score += x; goto BB6
BB5: score += 1; goto BB6
BB6: --x; goto BB2
BB7: return score

```



上述 goto-style pseudo IR 建立出的 CFG。

BB_4 和 BB_5 是不同的 branch targets，所以不能直接合併。

接下來的 BB_6 則是兩條分支重新接上的地方。它有 BB_4 、 BB_5 兩個 predecessors，因此不能把它和其中一邊直接合在一起。

最後， $BB_6 \rightarrow BB_2$ 又回到 loop header，形成迴圈。

支配關係 (Dominance)

在有 entry 的 CFG 中，如果從 entry 到 v 的每一條路徑都會經過 u ，就說 u 支配 v ，記作 $u \text{ dom } v$ 。

幾個基本性質：

- 每個節點都支配自己。
- 若 $u \neq v$ 且 u 支配 v ，就是 strict dominance。
- v 的 immediate dominator 記為 $\text{idom}(v)$ 。它是離 v 最近的 strict dominator。

以前面的七個 BB 為例：

節點	Dominators	idom
BB_1	$\{BB_1\}$	無
BB_2	$\{BB_1, BB_2\}$	BB_1
BB_3	$\{BB_1, BB_2, BB_3\}$	BB_2
BB_4	$\{BB_1, BB_2, BB_3, BB_4\}$	BB_3
BB_5	$\{BB_1, BB_2, BB_3, BB_5\}$	BB_3
BB_6	$\{BB_1, BB_2, BB_3, BB_6\}$	BB_3
BB_7	$\{BB_1, BB_2, BB_7\}$	BB_2

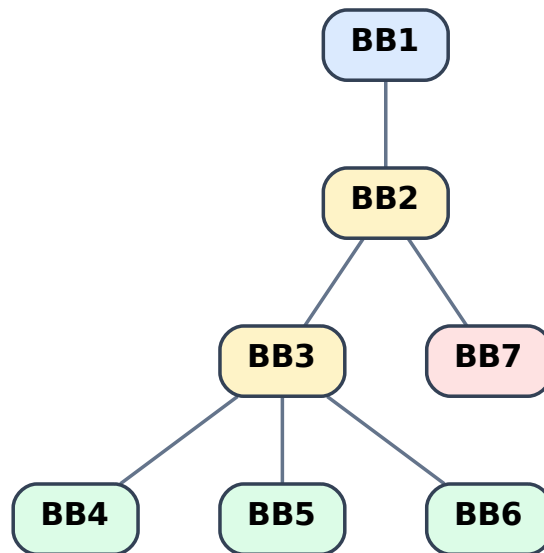
例如 BB_4 不支配 BB_6 。因為可以走 $BB_3 \rightarrow BB_5 \rightarrow BB_6$ ，完全不經過 BB_4 。

BB_5 也是一樣。

但不管走哪條 branch，都必須先經過 BB_3 ，所以 $\text{idom}(BB_6) = BB_3$ 。

Dominator Tree

把每個非 entry 節點 v 接到它的 $\text{idom}(v)$ ，就會得到 dominator tree：



完整範例的 dominator tree。父節點是子節點的 immediate dominator。

在 dominator tree 裡，如果 u 是 v 的 ancestor，就代表 u 支配 v 。

要注意的是，dominator tree 的 edge 不一定是原本 CFG 裡真的存在的 control-flow edge。它表示的是「哪些節點一定會先經過」這件事。

Dominator 會出現在很多 compiler analysis 裡。例如：

- 判斷 SSA value 的 definition 是否支配它的 uses；
- 配合 dominance frontier 決定 phi 要放在哪些 join points；
- 判斷某些運算能不能安全移動；
- 找出 loop header 和 natural loop。

Dominance 與 Loop

對一條 CFG edge $u \rightarrow h$ ，如果 h 支配 u ，這條 edge 就叫做 back edge。在一般的 reducible CFG 裡， h 通常就是 loop header。

在上面的完整範例中， BB_2 支配 BB_6 ，所以 $BB_6 \rightarrow BB_2$ 是 back edge。

這條 back edge 對應的 natural loop 是：

{ $BB_2, BB_3, BB_4, BB_5, BB_6$ }

BB_2 是 header, BB_6 是跳回 header 的 latch; $BB_2 \rightarrow BB_7$ 則是離開迴圈的 exit edge。

因此 BB_7 不屬於這個 loop。

一般有向圖裡的 cycle 不一定都有一個支配整個 cycle 的 header。這種 CFG 可以是 irreducible。

相較之下, while、for 這類結構化語法通常會產生比較容易分析的 reducible CFG。

回到本題

這題已經直接給出 CFG, 所以不需要自己把 C code 再 lowering 一次。

輸入中的每個點就是一個 BB, 每條有向邊代表一種可能的控制轉移。程式執行時, 就沿著 CFG 從 entry 1 往下走, 直到 exit N 。

題目中的性能計數器統計的, 就是這些 control-flow edges 實際被走過多少次。

115 學年度資訊學科能力競賽臺南一中校內複選 試題本

附錄

參考公式:

- 等加速度運動： $v = v_0 + at$ 、 $x = v_0t + \frac{1}{2}at^2$ 、 $v^2 = v_0^2 + 2ax$
- 牛頓第二運動定律： $F = ma$
- 萬有引力定律： $F = G\frac{m_1m_2}{r^2}$
- 動能與重力位能： $E_k = \frac{1}{2}mv^2$ 、 $E_p = mgh$ （地表附近）
- 庫侖定律： $F = k\frac{q_1q_2}{r^2}$
- 歐姆定律與電功率： $V = IR$ 、 $P = IV = I^2R = \frac{V^2}{R}$
- 波動基本公式： $v = f\lambda$
- 光電效應與光子能量： $E = hf$
- 質能互換公式： $E = mc^2$

祝各位都能破台，GL & HF